PART B
(PART B: TO BE COMPLETED BY STUDENTS)
Experiment 7

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

Roll.No: C-60	Name: Avdhut Madhavi
Class : SE-C	Batch: C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a PL/SQL statement and check the output.

TABLE:
Create the STUDENT table:

```
CREATE TABLE STUDENT (
   ROLL_NO NUMBER PRIMARY KEY,
   STUDENT_NAME VARCHAR2(50),
   MARKS NUMBER(5,2)
);
```

Insert The data into the table

```
INSERT INTO STUDENT VALUES (1, 'Aarav', 85);
INSERT INTO STUDENT VALUES (2, 'Diya', 42);
INSERT INTO STUDENT VALUES (3, 'Kabir', 32);

COMMIT;

SELECT * FROM STUDENT;
```

```
SQL> SELECT * FROM STUDENT;

   ROLL_NO STUDENT_NAME                                        MARKS
---------- -------------------------------------------------- ----------
         1 Aarav                                                  85
         2 Diya                                                   42
         3 Kabir                                                  32
```

Stored Procedures

1. Create a procedure to calculate the **square of a number**.

```
CREATE OR REPLACE PROCEDURE CALCULATE_SQUARE (
```

```
    p_num IN NUMBER
)
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Square = ' || (p_num * p_num));
END;
/


EXEC CALCULATE_SQUARE(5);
```

```
SQL> EXEC CALCULATE_SQUARE(5);
Square = 25

PL/SQL procedure successfully completed.
```

2. Create a procedure to calculate the **sum of two numbers**.

```
CREATE OR REPLACE PROCEDURE CALCULATE_SUM (
    p_num1 IN NUMBER,
    p_num2 IN NUMBER
)
AS
BEGIN
    DBMS_OUTPUT.PUT_LINE('Sum = ' || (p_num1 + p_num2));
END;
/

EXEC CALCULATE_SUM(10, 20);
```

```
SQL> EXEC CALCULATE_SUM(10, 20);
Sum = 30

PL/SQL procedure successfully completed.
```

3. Create a procedure to display **student details using Roll_No**.

```
CREATE OR REPLACE PROCEDURE DISPLAY_STUDENT (
```

```
    p_roll_no IN NUMBER
)
AS
    v_name STUDENT.STUDENT_NAME%TYPE;
    v_marks STUDENT.MARKS%TYPE;
BEGIN
    SELECT STUDENT_NAME, MARKS
    INTO v_name, v_marks
    FROM STUDENT
    WHERE ROLL_NO = p_roll_no;

    DBMS_OUTPUT.PUT_LINE('Roll No: ' || p_roll_no);
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Marks: ' || v_marks);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');
END;
    /

    EXEC DISPLAY_STUDENT(1);
```

```
SQL> EXEC DISPLAY_STUDENT(1);
Roll No: 1
Name: Aarav
Marks: 85

PL/SQL procedure successfully completed.
```

4. Create a procedure to display **PASS/FAIL for a student**.

```
CREATE OR REPLACE PROCEDURE CHECK_RESULT (
    p_roll_no IN NUMBER
)
AS
    v_marks STUDENT.MARKS%TYPE;
BEGIN
    SELECT MARKS
    INTO v_marks
```

```
    FROM STUDENT
    WHERE ROLL_NO = p_roll_no;

    IF v_marks >= 40 THEN
        DBMS_OUTPUT.PUT_LINE('PASS');
    ELSE
        DBMS_OUTPUT.PUT_LINE('FAIL');
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');
END;
/

EXEC CHECK_RESULT(1);
```

```
SQL> EXEC CHECK_RESULT(1);
PASS

PL/SQL procedure successfully completed.
```

5. Create a procedure to calculate the **area of a circle**.

```
CREATE OR REPLACE PROCEDURE CIRCLE_AREA (
    p_radius IN NUMBER
)
AS
    v_area NUMBER;
BEGIN
    v_area := 3.14159 * p_radius * p_radius;

    DBMS_OUTPUT.PUT_LINE('Area = ' || ROUND(v_area, 2));
END;
/

EXEC CIRCLE_AREA(7);
```

```
SQL> EXEC CIRCLE_AREA(7);
Area = 153.94

PL/SQL procedure successfully completed.
```

Stored Functions

6. Create a function to return the **square of a number**.

```
CREATE OR REPLACE FUNCTION SQUARE_FUNCTION (
   p_num IN NUMBER
)
RETURN NUMBER
AS
BEGIN
   RETURN p_num * p_num;
END;
/

SELECT SQUARE_FUNCTION(5) AS SQUARE
FROM DUAL;
```

```
SQL> SELECT SQUARE_FUNCTION(5) AS SQUARE
  2  FROM DUAL;

    SQUARE
----------
        25
```

7. Create a function to return the **sum of two numbers**.

```
CREATE OR REPLACE FUNCTION SUM_FUNCTION (
   p_num1 IN NUMBER,
   p_num2 IN NUMBER
)
RETURN NUMBER
AS
BEGIN
   RETURN p_num1 + p_num2;
END;
/

SELECT SUM_FUNCTION(10, 20) AS TOTAL
FROM DUAL;
```

```
SQL> SELECT SUM_FUNCTION(10, 20) AS TOTAL
  2  FROM DUAL;

     TOTAL
----------
        30
```

8. Create a function to return **PASS/FAIL** for a student.

```
CREATE OR REPLACE FUNCTION STUDENT_RESULT (
   p_roll_no IN NUMBER
)
RETURN VARCHAR2
AS
   v_marks STUDENT.MARKS%TYPE;
BEGIN
   SELECT MARKS
   INTO v_marks
   FROM STUDENT
   WHERE ROLL_NO = p_roll_no;

   IF v_marks >= 40 THEN
      RETURN 'PASS';
   ELSE
      RETURN 'FAIL';
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'STUDENT NOT FOUND';
END;
/


SELECT STUDENT_RESULT(1) AS RESULT
FROM DUAL;
```

```
SQL> SELECT STUDENT_RESULT(1) AS RESULT
  2  FROM DUAL;

RESULT
--------------------------------------------------------------------------------
PASS
```

9. Create a function to return the **grade of a student**.

```
CREATE OR REPLACE FUNCTION STUDENT_GRADE (
   p_roll_no IN NUMBER
)
RETURN VARCHAR2
AS
   v_marks STUDENT.MARKS%TYPE;
BEGIN
   SELECT MARKS
   INTO v_marks
   FROM STUDENT
   WHERE ROLL_NO = p_roll_no;
```

```
    IF v_marks >= 80 THEN
        RETURN 'A';
    ELSIF v_marks >= 60 THEN
        RETURN 'B';
    ELSIF v_marks >= 40 THEN
        RETURN 'C';
    ELSE
        RETURN 'F';
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'STUDENT NOT FOUND';
END;
/


SELECT STUDENT_GRADE(1) AS GRADE
FROM DUAL;
```

```
SQL> SELECT STUDENT_GRADE(1) AS GRADE
  2  FROM DUAL;

GRADE
--------------------------------------------------------------------------------
A
```

10. Create a function to return the **marks of a student using Roll_No**.

```
CREATE OR REPLACE FUNCTION GET_STUDENT_MARKS (
    p_roll_no IN NUMBER
)
RETURN NUMBER
AS
    v_marks STUDENT.MARKS%TYPE;
BEGIN
    SELECT MARKS
    INTO v_marks
    FROM STUDENT
    WHERE ROLL_NO = p_roll_no;

    RETURN v_marks;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/


SELECT GET_STUDENT_MARKS(1) AS MARKS
```

FROM DUAL;

```
SQL> SELECT GET_STUDENT_MARKS(1) AS MARKS
  2  FROM DUAL;

     MARKS
----------
        85
```

B.2: Write an answer of the following questions

1. What is a stored procedure?
   A stored procedure is a named PL/SQL block stored in the database that performs a specific task. It can accept parameters and execute SQL statements.

2. What is a stored function?
   A stored function is a named PL/SQL block that performs a specific task and returns a value using the RETURN statement.

3. What is the difference between a procedure and a function?

Procedure	Function
May or may not return values through parameters.	Must return a value using RETURN.
Executed using EXEC or a PL/SQL block.	Can be called in expressions and suitable SQL statements.
Mainly used to perform an operation.	Commonly used to calculate and return a value.

4. What is the purpose of CREATE OR REPLACE?
   It creates a procedure or function if it does not exist, or replaces the existing definition if it already exists.

5. What is an IN parameter?
   An IN parameter passes a value into a procedure or function. It is used for input and cannot be assigned a new value within the subprogram.

6. Why is RETURN required in a function?
   RETURN sends the result back to the calling statement. A function must return a value of its declared return type.

7. How is a procedure executed?
   A procedure can be executed using the EXEC command.
   Example:
   EXEC CALCULATE_SQUARE(5);

8. How is a function called?
   A function can be called in a SELECT statement or a PL/SQL expression.
   Example:
   SELECT SQUARE_FUNCTION(5)
   FROM DUAL;

9. What is SELECT...INTO?
   SELECT...INTO retrieves data from a query and stores the result in PL/SQL variables.
   Example:

   SELECT MARKS
   INTO v_marks
   FROM STUDENT
   WHERE ROLL_NO = 1;

10. What is the purpose of DUAL?
    DUAL is a special one-row table in Oracle used to evaluate expressions and call functions in a SELECT statement when data from another table is not required.

11. What is DBMS_OUTPUT.PUT_LINE?
    It displays text or values in the SQL*Plus output when server output is enabled.

12. Why is SET SERVEROUTPUT ON used?
    It enables the display of messages generated by DBMS_OUTPUT.PUT_LINE.

13. Can a function use data from a table?
    Yes. A function can retrieve table data using SQL statements such as SELECT...INTO, provided it follows Oracle's rules and restrictions.

14. Can a function be called in a SELECT statement?
    Yes. A function that is valid for SQL use can be called in a SELECT statement.
    Example:
    SELECT SQUARE_FUNCTION(5)
    FROM DUAL;